meso360 Field Guide

NSSL Mobile Mesonet Data System — Complete Reference

> This document covers system architecture, installation, configuration, dashboard operation, variable definitions, troubleshooting, and field deployment procedures.

Table of Contents

1. System Overview
    - What is meso360?
    - Components
    - Data Flow
2. System Requirements
3. Installation
    - Step 1 — Clone the Repository
    - Step 2 — Set Up the Python Environment
    - Step 3 — Configure
    - Step 4 — SSH Key Setup (mesosync)
4. Configuration Reference
5. First Run and Preflight
    - Starting the System
    - Preflight Checks
    - Resolving WARN Conditions
6. Dashboard Guide
    - Connection Status Dot
    - Header Controls
    - Charts
    - Sidebar: Heading
    - Sidebar: Current Values
    - Sidebar: 30-Second Averages
    - Sidebar: Max Wind Speed
    - Sidebar: Status Card
    - Update Indicator
7. Variables Reference
    - Sensor Variables
    - GPS Variables
    - Derived Variables
8. Data File Format
    - File Naming
    - File Structure
    - GPS Date and Time Format
    - Missing and Invalid Values
9. Run on Startup
    - macOS — launchd
    - Linux — systemd
    - Linux — cron

    - Windows — Task Scheduler
10. Updating
11. Troubleshooting
    - Dashboard Symptoms
    - Log Symptoms
    - Reading the Log File
12. File Layout
13. Appendix — Printable Preflight Checklist

---

1. System Overview

What is meso360?

meso360 is a real-time data acquisition and display system for NSSL Mobile Mesonet vehicles. It reads meteorological observations from the onboard Campbell Scientific datalogger at 1 Hz, writes them to daily plain-text files, and serves a live interactive web dashboard accessible from any device on the vehicle's local network.

The system is designed to run unattended. Once started, it manages its own process lifecycle — restarting crashed components, rotating log files at midnight, and maintaining a remote SSH connection for off-site access.

Components

Component	Script	Role
mesoingest	`mesoingest.py`	Fetches observations from the datalogger at 1 Hz and writes them to daily `.txt` files
mesoview	`mesoview.py`	Serves the web dashboard and streams live data to browsers over SSE
mesosync	*(managed by supervisor)*	Maintains a persistent SSH reverse tunnel to `remote.bliss.science` for remote access
supervisor	`supervisor.py`	Starts and monitors all three components; restarts any that crash; rotates logs at midnight

You only ever need to start `supervisor.py`. It manages everything else.

Data Flow

```
Campbell Scientific Datalogger
        │  HTTP (LAN, 1 Hz)
        ▼
  mesoingest.py
        │  appends CSV rows
        ▼
  ~/data/raw/mesonet/YYYYMMDD.txt
        │  tailed at 1 Hz
        ▼
  mesoview.py  ─── SSE (live JSON) ──► Browser dashboard
```

```
          │
          └── /initial (JSON, last 2 hr) ──► Browser on page load

    supervisor.py ── SSH reverse tunnel ──► remote.bliss.science
    (mesosync)
```

All log output from all three components flows through the supervisor into a shared daily log file at `~/mesoview_logs/mesoview.YYYYMMDD.log`.

---

2. System Requirements

Hardware

- Any laptop or mini-PC running macOS, Linux, or Windows 10/11
- Minimum 4 GB RAM (16 GB recommended; the in-memory data cache uses ~50 MB at 1 Hz over 2 hours)
- Network connection to the Campbell Scientific datalogger (vehicle LAN)

Software

Requirement	Notes
Python 3.10 or later	Via Miniforge/conda (recommended) or any standard Python installation
Miniforge	Recommended Python distribution; provides conda for environment management
OpenSSH client	Built into macOS, Linux, and Windows 10/11 — no WSL or extra tools needed
`~/.ssh/clamps_rsa`	NSSL/BLISS RSA key — required for mesosync to connect; see SSH Key Setup

Python Dependencies

Managed automatically by the conda environment or pip:

Package	Used by
`flask`	mesoview — web server and SSE streaming
`requests`	mesoingest — HTTP requests to the datalogger
`zeroconf`	mesoview — mDNS/Bonjour `.local` hostname advertisement (optional; falls back gracefully if missing)

---

3. Installation

Step 1 — Clone the Repository

```
git clone <repo-url>
cd meso360
```

Step 2 — Set Up the Python Environment

Option A — conda (recommended)

Conda creates an isolated environment that won't conflict with other Python installations on the machine.

```
conda env create -f environment.yml
conda activate mesoview
```

To update the environment after a `git pull` that adds new dependencies:

```
conda env update -f environment.yml --prune
```

Option B — pip / venv

```
python -m venv .venv

# macOS / Linux
source .venv/bin/activate

# Windows
.venv\Scripts\activate

pip install -r requirements.txt
```

> **Note:** The virtual environment must be activated every time you open a new terminal before running the system. The startup scripts in Section 9 reference the Python executable directly so they work without manual activation.

Step 3 — Configure

Copy the example config and edit it for this vehicle:

```
cp mesoview.config.example.json mesoview.config.json
```

`mesoview.config.json` is listed in `.gitignore` and will never be overwritten by a `git pull`. Open it in any text editor and set the values for this vehicle. See Section 4 — Configuration Reference for a full description of every key.

Minimum required changes:

- Set `logger_ip` to the LAN IP of the Campbell datalogger on this vehicle's network
- Set `rtun_port` to this vehicle's unique SSH tunnel port (assigned by NSSL/BLISS — check with the team if unknown)

Step 4 — SSH Key Setup (mesosync)

mesosync uses a shared RSA key (`clamps_rsa`) to authenticate the SSH reverse tunnel. This key must be present on each vehicle machine before the tunnel will connect.

1. Obtain `clamps_rsa` from the NSSL/BLISS team (it is not stored in this repository)
2. Copy it to the correct location:

```
# macOS / Linux
mkdir -p ~/.ssh
cp /path/to/clamps_rsa ~/.ssh/clamps_rsa
chmod 600 ~/.ssh/clamps_rsa   # SSH requires the key file to be owner-readable
only

# Windows (PowerShell)
mkdir $HOME\.ssh -Force
Copy-Item C:\path\to\clamps_rsa $HOME\.ssh\clamps_rsa
```

> **Without this key**, mesosync will fail to connect and supervisor will restart it repeatedly. This does not affect mesoingest or mesoview — data collection and the dashboard work independently of the tunnel.

---

4. Configuration Reference

All settings live in `mesoview.config.json`. Every key is optional; the system falls back to the default shown below if a key is missing.

```
{
  "data_dir":           "~/data/raw/mesonet",
  "log_dir":            "~/mesoview_logs",
  "logger_ip":          "192.168.4.6",
  "http_port":          8080,
  "mdns_hostname":      "mesoview",
  "ingest_retry_max":   100,
  "ingest_retry_delay": 5,
  "rtun_port":          2222
}
```

Key	Type	Default	Component	Description
`data_dir`	string (path)	`~/data/raw/mesonet`	mesoingest, mesoview	Directory where daily `.txt` data files are written by mesoingest and read by mesoview. Supports `~` for home directory. Created automatically on first run.

Key	Type	Default	Component	Description
`log_dir`	string (path)	`~/mesoview_logs`	supervisor	Directory where daily log files are written. File names follow the pattern `mesoview.YYYYMMDD.log`. Created automatically on first run.
`logger_ip`	string	`192.168.4.6`	mesoingest	LAN IP address of the Campbell Scientific datalogger. Check the vehicle network configuration if the default does not work.
`http_port`	integer	`8080`	mesoview	Port the dashboard web server listens on. Change if 8080 conflicts with another service on this machine.
`mdns_hostname`	string	`mesoview`	mesoview	Hostname advertised via mDNS/Bonjour. The dashboard will be accessible at `http://<hostname>.local:<port>` on networks that support mDNS.
`ingest_retry_max`	integer	`100`	mesoingest	Maximum number of consecutive fetch failures before mesoingest exits (allowing supervisor to restart it). At the default retry delay of 5 s, this gives ~8 minutes of retries before a restart.
`ingest_retry_delay`	integer	`5`	mesoingest	Seconds to wait between retry attempts when the datalogger is unreachable.
`rtun_port`	integer	*(none)*	mesosync	Port for the SSH reverse tunnel to `remote.bliss.science`. **Must be unique per vehicle** — two vehicles using the same port will conflict. If omitted, mesosync is disabled entirely.

> **Config changes require a restart** of `supervisor.py` to take effect. The config is read once at startup.

---

5. First Run and Preflight

Starting the System

Activate your environment, then run the supervisor from the repo directory:

```
conda activate mesoview       # or: source .venv/bin/activate
cd /path/to/meso360
python supervisor.py
```

The supervisor will start mesoingest, mesoview, and mesosync (if configured), writing all output to both the terminal and the daily log file.

Open a browser on any device on the same network and navigate to:

```
http://<host-machine-ip>:8080
```

Or, on networks with mDNS/Bonjour support:

```
http://mesoview.local:8080
```

Preflight Checks

Before launching any child processes, `supervisor.py` runs a set of startup checks and logs the results. This output appears near the top of the log file (and the terminal if running interactively):

```
[2025-06-05 17:52:28] [supervisor] === Preflight checks ===
[2025-06-05 17:52:28] [supervisor]   PASS  config file found:
/path/to/meso360/mesoview.config.json
[2025-06-05 17:52:28] [supervisor]   PASS  data directory writable:
/home/user/data/raw/mesonet
[2025-06-05 17:52:28] [supervisor]   PASS  SSH key found:
/home/user/.ssh/clamps_rsa
[2025-06-05 17:52:28] [supervisor] ========================
```

All three checks should show `PASS` before a field deployment. A `WARN` indicates something that needs attention — the system will still start, but the affected component may not function correctly.

Resolving WARN Conditions

Config file not found

```
WARN  config file not found: .../mesoview.config.json
      Run: cp mesoview.config.example.json mesoview.config.json
```

What it means: The system is running on default values. The datalogger IP and SSH tunnel port are almost certainly wrong.

Fix: Run the `cp` command shown, then edit `mesoview.config.json` with the correct values for this vehicle. Restart supervisor.

Data directory not writable

```
WARN  data directory not writable: /path/to/data/dir (Permission denied)
      Fix permissions or set "data_dir" in mesoview.config.json
```

What it means: mesoingest will fail to write data files. No data will be collected until this is resolved.

Fix (Linux/macOS):

```
# Option 1 — fix permissions on the existing directory
chmod 755 /path/to/data/dir

# Option 2 — point to a directory the current user owns
# Edit mesoview.config.json:  "data_dir": "~/data/raw/mesonet"
```

Restart supervisor after fixing.

SSH key not found

```
WARN  SSH key not found: /home/user/.ssh/clamps_rsa
      Copy clamps_rsa to that path — mesosync will not connect until this is
done
```

What it means: mesosync will fail to establish the reverse tunnel. Data collection and the dashboard are unaffected.

Fix: See Step 4 — SSH Key Setup. The key must be in place and have permissions `600` before mesosync will connect. Supervisor will automatically pick it up on the next mesosync restart (within a few seconds) — no full restart required once the key is in place.

---

6. Dashboard Guide

The dashboard is a single-page web application served by mesoview. It shows live data streamed from the host machine over SSE (Server-Sent Events) — a persistent connection that pushes one update per second to every connected browser.

Connection Status Dot

The small colored circle in the top-left corner of the header shows the health of the data connection:

Color	State	Meaning
Green	Live	SSE connection open and data arriving normally
Orange	Stale	SSE connection open but no data received in the last 30 seconds. mesoingest may be down, the datalogger may be unreachable, or the instrument rack may not be powered on.
Red	Disconnected	SSE connection to mesoview lost. mesoview may have crashed, or the network between this browser and the host machine is interrupted. The browser reconnects automatically every 5 seconds.

The timestamp next to the dot also turns orange when data is stale.

Header Controls

Control	Function
Timestamp	UTC time of the last received data point. Turns orange when stale.
Pause / Live	Pauses the chart from advancing with new data so you can inspect a moment in time. Click again (or double-click a chart) to resume live mode. While paused, data continues to be buffered in the background.
Reset Zoom	Appears when you have drag-zoomed or scrolled a chart. Resets all charts back to the live rolling window.

Chart navigation:

Action	Desktop	Mobile
Zoom in	Click and drag on the chart	Pinch
Pan	Scroll horizontally / trackpad swipe	Single-finger swipe
Reset zoom	Double-click	Double-tap

Charts

Three charts fill the left column of the desktop layout (stacked vertically). On mobile, the combined wind chart splits into two separate charts.

Temperature and Dewpoint

Plots `t_fast` (temperature, red) and `dewpoint` (dewpoint, blue) on a shared °C axis. Both lines break at data gaps of ≥ 2 seconds.

Wind Speed and Direction

- **Desktop:** Combined chart with wind speed (m/s, green, left axis) and wind direction (degrees, orange dots, right axis). Direction is plotted as individual dots rather than a connected line — connecting direction through e.g. 350° → 10° would falsely sweep through 180° (south).
- **Mobile:** Two separate charts, one for speed and one for direction.

Pressure

Station pressure (hPa, purple). Y-axis auto-scales to the visible data range.

Sidebar: Heading

Canvas compass rose showing the vehicle's current heading from the magnetic compass sensor (`compass_dir`). The orange arrow points in the direction the vehicle is facing. The numeric heading is shown below the rose in degrees.

Sidebar: Current Values

Instantaneous readings from the most recently received data point.

Field	Variable	Units
T	`t_fast`	°C / °F
Td	`dewpoint`	°C / °F
Pressure	`pressure`	hPa
Speed	`sfc_wspd`	m/s
Wind Dir	`sfc_wdir`	degrees
θe	Derived (Bolton 1980)	K

Sidebar: 30-Second Averages

Rolling averages over the last 30 observations (30 seconds at 1 Hz). Wind direction is averaged using circular mean (vector arithmetic) to correctly handle wrap-around near 0°/360°. All other fields use arithmetic mean with null values excluded.

Field	Variable	Notes
T	`t_fast`	°C / °F
Td	`dewpoint`	°C / °F
Pressure	`pressure`	hPa
Speed	`sfc_wspd`	m/s
Wind Dir	`sfc_wdir`	Circular mean
θe	Derived	K

Sidebar: Max Wind Speed

Tracks the highest wind speed (`sfc_wspd`) observed since the start of the current operations day (defined as the first data point after a gap of ≥ 4 hours). Also records the time and GPS position of the peak gust.

Click **Reset Max** to clear the recorded peak. The tracker will then begin recording anew from the next incoming observation.

Field	Description
Speed	Peak `sfc_wspd` in m/s
Time	UTC time of the peak
Lat	GPS latitude at the time of peak
Lon	GPS longitude at the time of peak

Sidebar: Status Card

The Status card at the bottom of the sidebar provides a compact system health summary.

Field	Description	Notes
Last update	Time of the most recently received data point	Local time format
Gaps (UTC day)	Number of data gaps (≥ 2 s between readings) since UTC midnight	Resets at midnight
Completeness	Percentage of expected 1 Hz observations received in the last 10 minutes	100% = no missed readings
3-min ΔP	Pressure change over the last 3 minutes (hPa)	Positive = rising, negative = falling
GPS fix	**Fix** (green) = valid GPS position; **No Fix** (orange) = GPS unit reporting `nan`	No Fix does not stop data collection; UTC time is used as a fallback for file naming
Record age	Seconds since the GPS timestamp of the latest record	Turns orange when > 30 s, matching the header connection dot

Update Indicator

In the bottom-right corner of the dashboard, a small indicator shows the git status of the running code:

State	Appearance	Meaning
Up to date	Hidden	Nothing to do
Update available	`↑ Update · <commit>` button	Remote repository has new commits. Click to run `git pull` and restart mesoview automatically.
DEV badge	Orange `DEV` badge	Repo has uncommitted local changes. The Update button is hidden to avoid overwriting local work.

7. Variables Reference

Sensor Variables

These columns are written directly from the Campbell datalogger to the daily `.txt` file.

Column	Full Name	Units	Description	Typical Range
`sfc_wspd`	Surface wind speed	m/s	Wind speed from the RM Young sonic anemometer	0 – 40 m/s
`sfc_wdir`	Surface wind direction	degrees (0–360)	Wind direction from the anemometer; 0°/360° = North, 90° = East	0 – 360°
`t_slow`	Temperature (slow)	°C	Slow-response temperature sensor	−40 – 60°C
`rh_slow`	Relative humidity (slow)	%	Slow-response RH sensor	0 – 100%
`t_fast`	Temperature (fast)	°C	Fast-response temperature sensor; used for displayed T	−40 – 60°C
`dewpoint`	Dewpoint	°C	Dewpoint temperature from the humidity sensor	−40 – 35°C
`der_rh`	Derived RH	%	Relative humidity derived from `t_fast` and `dewpoint`	0 – 100%
`pressure`	Station pressure	hPa	Raw station (not sea-level) pressure; varies with elevation	850 – 1050 hPa
`compass_dir`	Compass heading	degrees (0–360)	Vehicle heading from the magnetic compass; used for the dashboard rose	0 – 360°
`panel_temp`	Panel temperature	°C	Instrument enclosure/panel temperature; diagnostic use	−20 – 60°C

> **`t_fast` vs `t_slow`:** The fast-response sensor is preferred for meteorological use because it responds more quickly to ambient temperature changes as the vehicle moves. `t_slow` is a secondary reference.

> **`der_rh` vs `rh_slow`:** `der_rh` is derived from the fast-response temperature and dewpoint; `rh_slow` is measured directly by the slow-response humidity sensor. Both are available for cross-checking.

GPS Variables

GPS fields are populated by the onboard GPS receiver. When the GPS has no fix, all GPS fields report `nan` in the raw datalogger output.

Column	Format	Units	Description
`gps_date`	DDMMYY	—	GPS date; zero-padded to 6 digits (e.g., `050625` = June 5, 2025)

Column	Format	Units	Description
`gps_time`	HHMMSS	UTC	GPS time; zero-padded to 6 digits (e.g., `175228` = 17:52:28 UTC)
`lat`	Decimal degrees	degrees N	GPS latitude; positive = North
`lon`	Decimal degrees	degrees E	GPS longitude; negative = West (U.S. longitudes are negative)
`gps_alt`	—	meters MSL	GPS altitude above mean sea level
`gps_spd`	—	m/s	GPS-derived ground speed
`gps_dir`	degrees (0–360)	—	GPS-derived track direction (direction of travel, not vehicle heading)

> **GPS date vs UTC clock:** The daily data file is named using the GPS date (`YYYYMMDD.txt`). If the GPS has no fix at startup, mesoingest falls back to the UTC system clock for file naming and logs a `WARNING`. Once the GPS acquires a fix, normal GPS-based naming resumes. Observations written under the UTC-fallback file name are not retroactively moved.

Derived Variables

These variables are computed in the dashboard from the sensor data and are not written to the data files.

Equivalent Potential Temperature (θe)

Computed using Bolton (1980), Equations 10, 15, and 38. Inputs: `t_fast` (°C), `dewpoint` (°C), `pressure` (hPa). Output: θe in Kelvin.

θe represents the temperature a parcel would have if all its moisture were condensed out and then brought dry-adiabatically to 1000 hPa. It is conserved under both dry and moist adiabatic processes and is useful for identifying air mass boundaries and assessing convective potential.

Typical values during warm-season field operations in the southern Great Plains: 320 – 370 K. Values above 350 K are associated with high-CAPE environments.

---

8. Data File Format

File Naming

Daily data files are named `YYYYMMDD.txt` and stored in `data_dir` (default: `~/data/raw/mesonet`). The date in the filename comes from the GPS timestamp of the first observation of that day.

Example: `20250605.txt` contains all observations for UTC June 5, 2025.

File Structure

Each file is a plain-text CSV with a single header row followed by one observation row per second:

```
sfc_wspd,sfc_wdir,t_slow,rh_slow,t_fast,dewpoint,der_rh,pressure,compass_dir,g
ps_date,gps_time,lat,lon,gps_alt,gps_spd,gps_dir,panel_temp
2.548,108.8,23.1,82.2,23.2,19.92,81.8,900.85,93.1,050625,175228,33.19017,-102.
2727,1021,0,85.5,28.16
3.136,99.1,23.1,82.2,23.18,19.92,81.9,900.85,92.7,050625,175229,33.19017,-102.
2727,1021,0,85.5,28.16
```

- Columns are comma-separated, no spaces
- No units in the file — see Variables Reference for units
- Rows are appended at approximately 1 Hz
- The header is written once, on first creation of the file

GPS Date and Time Format

GPS date (`gps_date`) is in **DDMMYY** format, zero-padded to 6 digits:

Raw value	Parsed as
`050625`	June 5, 2025
`310525`	May 31, 2025
`010125`	January 1, 2025

GPS time (`gps_time`) is in **HHMMSS** UTC format, zero-padded to 6 digits:

Raw value	Parsed as
`175228`	17:52:28 UTC
`000001`	00:00:01 UTC
`120000`	12:00:00 UTC

Missing and Invalid Values

Condition	Raw value from datalogger	Behavior
GPS no fix	`nan` (string)	`gps_date`, `gps_time`, `lat`, `lon`, `gps_alt`, `gps_spd`, `gps_dir` all written as `nan`. mesoingest falls back to UTC clock for file naming and logs a WARNING. mesoview treats `nan` numeric fields as missing (no GPS dot on dashboard).
Sensor dropout	Field-specific; non-finite float	mesoview converts `inf` and `nan` floats to `null` in the dashboard buffer. The affected field shows `--` in sidebar cards and produces a gap in the chart.

Condition	Raw value from datalogger	Behavior
Datalogger unreachable	*(no data written)*	mesoingest retries indefinitely (up to `ingest_retry_max`), logging each failure. mesoview connection dot turns orange after 30 seconds with no new data.

9. Run on Startup

To have `supervisor.py` launch automatically at boot, first find the full path to the Python executable in your environment:

```
# conda
conda activate mesoview
which python          # macOS / Linux  → e.g.
/Users/you/miniforge3/envs/mesoview/bin/python
where python          # Windows        → e.g.
C:\Users\you\miniforge3\envs\mesoview\python.exe

# venv
# macOS / Linux: /path/to/meso360/.venv/bin/python
# Windows:       C:\path\to\meso360\.venv\Scripts\python.exe
```

---

macOS — launchd

Create `~/Library/LaunchAgents/com.mesoview.plist`:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>Label</key>             <string>com.mesoview</string>
  <key>ProgramArguments</key>
  <array>
    <string>/Users/YOUR_USER/miniforge3/envs/mesoview/bin/python</string>
    <string>/path/to/meso360/supervisor.py</string>
  </array>
  <key>WorkingDirectory</key>  <string>/path/to/meso360</string>
  <key>RunAtLoad</key>         <true/>
  <key>KeepAlive</key>         <true/>
</dict>
</plist>
```

Replace `YOUR_USER`, the Python path, and the repo path, then load the service:

```
launchctl load ~/Library/LaunchAgents/com.mesoview.plist
```

To stop or unload:

```
launchctl unload ~/Library/LaunchAgents/com.mesoview.plist
```

To check if it is running:

```
launchctl list | grep com.mesoview
```

Linux — systemd

Create `~/.config/systemd/user/mesoview.service`:

```
[Unit]
Description=meso360 supervisor
After=network.target

[Service]
WorkingDirectory=/path/to/meso360
ExecStart=/path/to/conda/envs/mesoview/bin/python supervisor.py
Restart=on-failure
RestartSec=5

[Install]
WantedBy=default.target
```

Enable and start:

```
systemctl --user enable mesoview
systemctl --user start mesoview
```

Useful management commands:

```
systemctl --user status mesoview    # check if running, recent log output
systemctl --user stop mesoview      # stop the service
systemctl --user restart mesoview   # restart (e.g. after a config change)
journalctl --user -u mesoview -f    # follow the systemd journal live
```

Linux — cron

A simpler alternative to systemd for machines that don't need full service management:

```
crontab -e
```

Add this line (replace paths):

```
@reboot cd /path/to/meso360 && /path/to/conda/envs/mesoview/bin/python
supervisor.py >> ~/mesoview_logs/cron_boot.log 2>&1
```

> **Note:** cron does not restart the process if it crashes. If crash recovery is important, prefer systemd.

Windows — Task Scheduler

1. Open **Task Scheduler** → **Create Task**
2. **General** tab: give it a name (e.g. `Mesoview`); check *Run whether user is logged on or not*
3. **Triggers** tab: New → *At startup*
4. **Actions** tab: New →
   - **Program/script**: full path to your env's Python executable, e.g. `C:\Users\YOU\miniforge3\envs\mesoview\python.exe`
   - **Add arguments**: `supervisor.py`
   - **Start in**: full path to the repo directory, e.g. `C:\path\to\meso360`
5. **Settings** tab: check *If the task fails, restart every 1 minute*
6. Click OK and enter your Windows password when prompted

To test without rebooting: right-click the task → **Run**.

To check that it is running: open **Task Manager** → **Details** → look for `python.exe`.

---

10. Updating

Using the Dashboard

If the remote repository has new commits, an `↑ Update · <commit>` button appears in the bottom-right corner of the dashboard. Clicking it:

1. Runs `git pull` in the repo directory
2. Shows the git output on screen
3. If new commits were downloaded, automatically restarts mesoview within a few seconds
4. The browser reloads once mesoview is back up

> **Note:** The dashboard update button only restarts mesoview. If `supervisor.py` or `mesoingest.py` changed, restart the full supervisor manually for those changes to take effect.

Manual Update

```
git pull
conda env update -f environment.yml --prune   # pick up any new dependencies
```

Then restart supervisor:

```
# If running manually:
Ctrl-C to stop, then: python supervisor.py

# If running via systemd:
systemctl --user restart mesoview

# If running via launchd:
launchctl unload ~/Library/LaunchAgents/com.mesoview.plist
launchctl load   ~/Library/LaunchAgents/com.mesoview.plist
```

The config file (`mesoview.config.json`) is never modified by a git update.

---

11. Troubleshooting

Dashboard Symptoms

Symptom	Likely Cause	Resolution
Connection dot is **red**	mesoview has crashed, or the network between browser and host machine is down	Wait — browser reconnects every 5 s. If it stays red, check that `supervisor.py` is still running on the host machine.
Connection dot is **orange**	Data stopped flowing — mesoingest may be retrying, the datalogger may be unreachable, or the rack is not powered on	Check the log file for `WARNING` messages from mesoingest. Verify the rack is powered and connected to the vehicle LAN.
GPS fix: No Fix in Status card	GPS receiver has not yet acquired satellites, or GPS antenna is blocked/disconnected	Normal at startup — typically resolves within a minute outdoors. If it persists, check the GPS antenna connection on the rack. Data collection continues using UTC system clock as a fallback.
Record age increasing rapidly	Same as orange dot — mesoingest has stopped writing new data	See orange dot resolution above.
Charts show a flat line or no data	mesoview started before any data files exist, or the data file is in a different directory than `data_dir`	Check `data_dir` in config matches where mesoingest is writing. Check the log for file path messages.
Dashboard shows old data and doesn't update	Browser SSE connection dropped silently	Refresh the page. If the problem recurs, check browser console for errors.
Completeness well below 100%	Intermittent datalogger connectivity, or LAN congestion	Review the log for retry messages. If completeness drops below ~95% consistently, check the vehicle LAN and datalogger power.

Log Symptoms

Log message	Meaning	Action
`[mesoingest] Read attempt N failed (...). Retrying in 5s...`	Datalogger is temporarily unreachable	Normal if rack just powered on — wait. If it continues, verify network and datalogger IP.
`[mesoingest] Failed to read data after 100 attempts. Terminating.`	Exhausted all retries; mesoingest exited	Supervisor will restart it. Investigate network/datalogger.
`[mesoingest] WARNING: GPS date invalid ('nan'), using UTC date YYYYMMDD`	GPS has no fix; UTC date used for file naming	Check GPS antenna. Normal at startup, should clear once moving outdoors.
`[mesoview] WARNING: still waiting for data file .../YYYYMMDD.txt (Xs elapsed)`	mesoview is waiting for mesoingest to create today's data file	Appears if mesoview starts before mesoingest. Resolves automatically once mesoingest writes its first record.
`[mesoview] WARNING: parse_row failed: ...`	A data row from the file could not be parsed	Indicates a malformed row in the data file. Usually a one-off; if it repeats on every row, check the datalogger output format.
`[supervisor] mesoingest exited with N; restarting in 2s`	mesoingest crashed or exited	Supervisor will restart it automatically. Check the log for the error message mesoingest printed before exiting.
`[supervisor] WARN SSH key not found: ~/.ssh/clamps_rsa`	SSH key missing	See SSH Key Setup.

Reading the Log File

All three components write to a shared daily log file. To follow it live:

```
tail -f ~/mesoview_logs/mesoview.$(date +%Y%m%d).log
```

Each line is prefixed with a UTC timestamp and the component name:

```
[2025-06-05 17:52:28] [supervisor] starting — log: .../mesoview.20250605.log
[2025-06-05 17:52:28] [supervisor] === Preflight checks ===
[2025-06-05 17:52:29] [mesoingest] Read attempt 1 failed (Connection refused).
Retrying in 5s...
[2025-06-05 17:52:34] [mesoingest] Read attempt 2 failed (Connection refused).
Retrying in 5s...
[2025-06-05 17:52:39] [mesoview] Starting MM Viewer — open
http://192.168.4.10:8080
```

Log files rotate at midnight UTC. The previous day's log remains at `mesoview.YYYYMMDD.log` (the previous date's stamp).

---

12. File Layout

```
meso360/
├── supervisor.py                   # Entry point — starts and monitors all
three components
├── mesoingest.py                   # Fetches observations from the Campbell
datalogger at 1 Hz
├── mesoview.py                     # Flask web server; streams live data and
serves the dashboard
├── mesoview.config.json            # Local config — git-ignored, never
overwritten by git pull
├── mesoview.config.example.json    # Template config — copy this to
mesoview.config.json
├── environment.yml                 # conda environment spec (Python +
dependencies)
├── requirements.txt                # pip fallback dependency list
├── templates/
│   └── index.html                  # Single-page dashboard (HTML/CSS/JS)
├── static/
│   ├── uplot.min.js                # uPlot charting library — auto-downloaded
on first run
│   └── uplot.min.css               # uPlot stylesheet — auto-downloaded on
first run
├── test_data/
│   └── test.txt                    # Sample data file for --test mode (no
datalogger needed)
└── docs/
    └── meso360_field_guide.md      # This document
```

Auto-generated at runtime (not in repo):

```
~/data/raw/mesonet/
│   └── YYYYMMDD.txt    # Daily observation files written by mesoingest

~/mesoview_logs/
    └── mesoview.YYYYMMDD.log    # Daily log files written by supervisor
```

---

13. Appendix — Printable Preflight Checklist

> Print this page and keep it with the vehicle. Work through it in order before each field deployment.

Pre-Departure Checklist

Vehicle and hardware

- ☐ Instrument rack powered on
- ☐ Vehicle LAN active and datalogger reachable (ping `192.168.4.6` or configured IP)
- ☐ GPS antenna connected and has clear sky view (or will have one at deployment site)

Host machine

- ☐ `mesoview.config.json` exists in the meso360 directory
- ☐ `logger_ip` in config matches the datalogger's LAN IP for this vehicle
- ☐ `rtun_port` in config is set to this vehicle's unique assigned port
- ☐ `~/.ssh/clamps_rsa` is present (`ls ~/.ssh/clamps_rsa` — should not say "No such file")

Starting the system

- ☐ Conda environment activated: `conda activate mesoview`
- ☐ Supervisor started: `python supervisor.py` (or verified running via launchd/systemd)
- ☐ Log shows all preflight checks as `PASS` — no `WARN` lines
- ☐ Dashboard opens in browser at `http://<host-ip>:8080`

Dashboard verification

- ☐ Connection dot is **green**
- ☐ Timestamp is updating every second
- ☐ Temperature and pressure values look physically reasonable
- ☐ GPS fix shows **Fix** (green) — or **No Fix** is expected to clear once outdoors
- ☐ Record age is < 5 seconds

If any item above fails, refer to Section 11 — Troubleshooting or check the log file:

```
tail -f ~/mesoview_logs/mesoview.$(date +%Y%m%d).log
```

---

meso360 — NSSL Mobile Mesonet Data System